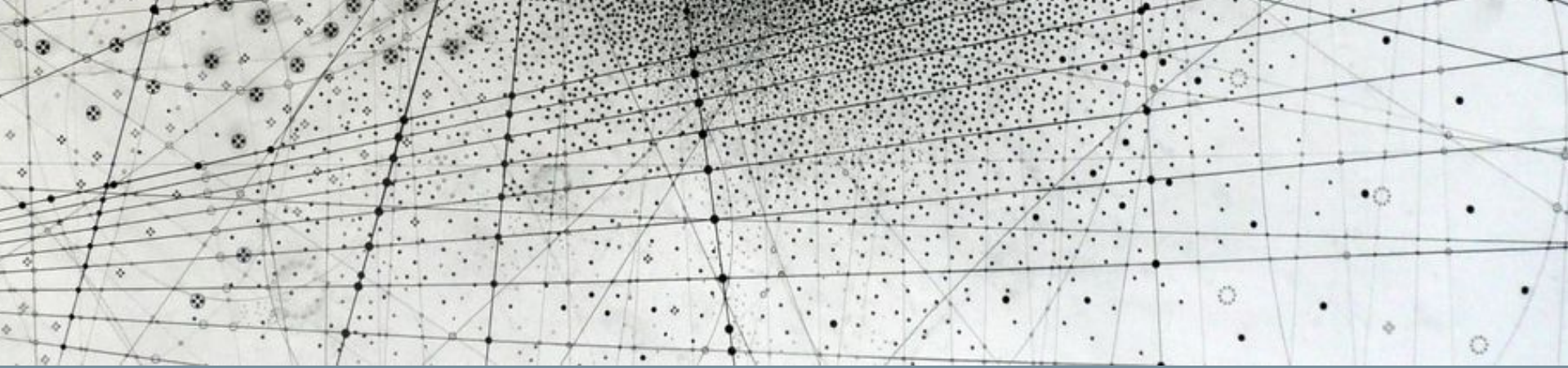




Research Methods in Machine Learning & Physics

Quick recap

Quick Recap: Activations & Optimizers

- **Activation functions help give the firings of our neurons meaning.**
 - e.g. "how positive is this neuron? "can i interpret this number as a probability?"
- Activation functions also **introduce nonlinearities** into our neural networks.
 - The more activations = the more expressive your network can be
- But the choice matters:
 - Sigmoid/tanh activations in the hidden layers leads to vanishing gradients
 - The ReLU family is a good default choice for hidden layers
 - Your final activation depends on your task
- **Optimizers are the "coaches" that help your network strategize how to learn.**
 - Full-batch gradient descent (GD): most thorough, but can get stuck in local minima
 - Stochastic gradient descent (SGD): train on mini-batches of data; efficient & nimble
 - Momentum: weighted average of previous steps, but fixed step-size
 - Adam: one of many optimizers that combines Momentum with adaptive step sizes

Adam, round 2



Anyone knows adam?

"dimension"?

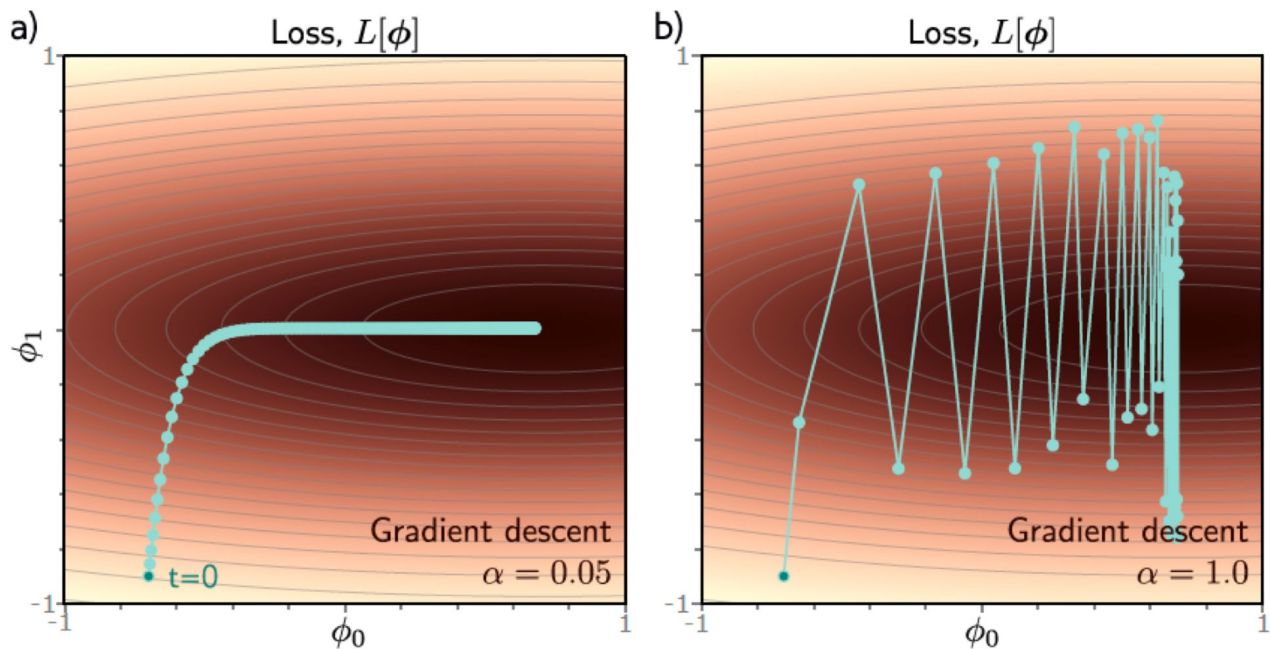
- I. 336: "Both architectures are optimized with Adam Who/what is "Adam"? I think this is a very serious typo that the author should have removed from the submission.

8:04 PM · Jul 24, 2025 · **618.9K** Views



Adam, round 2

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes *per parameter*



Adam, round 2

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes *per parameter*

```
class torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0,
amsgrad=False, *, foreach=None, maximize=False, capturable=False, differentiable=False,
fused=None, decoupled_weight_decay=False)
```

Momentum

Update the weights:

$$w_{t+1} = w_t - \alpha m_t$$

where:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

RMSProp

Update the weights:

$$w_{t+1} = w_t - \frac{\alpha_t}{\sqrt{v_t + \epsilon}} \frac{\partial L}{\partial w_t}$$

where:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

Adam, round 2

- **Adam**, i.e. "adaptive moment estimation", adapts the step sizes *per parameter*

```
class torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0,
amsgrad=False, *, foreach=None, maximize=False, capturable=False, differentiable=False,
fused=None, decoupled_weight_decay=False)
```

- **First moment (mean) estimate:**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

- **Second moment (variance) estimate:**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

Corrections that get smaller over time:

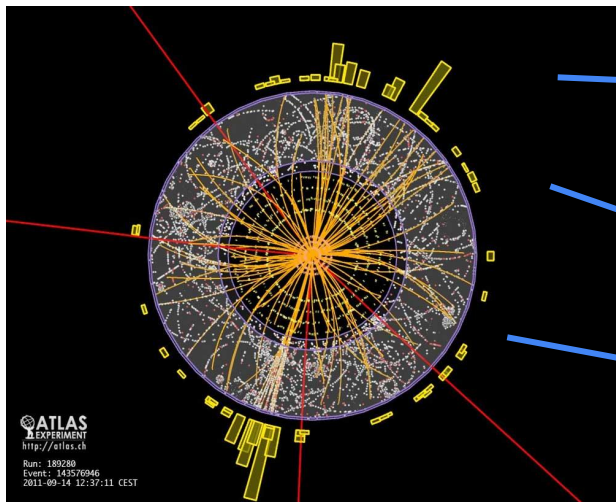
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \alpha$$

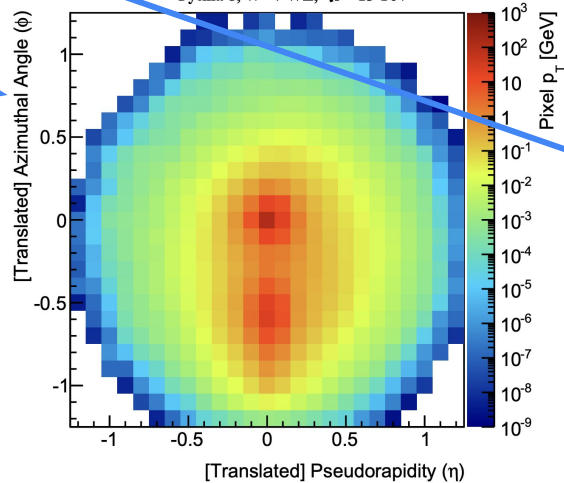
Geometric Deep Learning

Geometric deep learning



$250 < p_T/\text{GeV} < 260 \text{ GeV}$, $65 < \text{mass}/\text{GeV} < 95$

Pythia 8, $W' \rightarrow WZ$, $\sqrt{s} = 13 \text{ TeV}$



Reconstructed hadronic tau decay

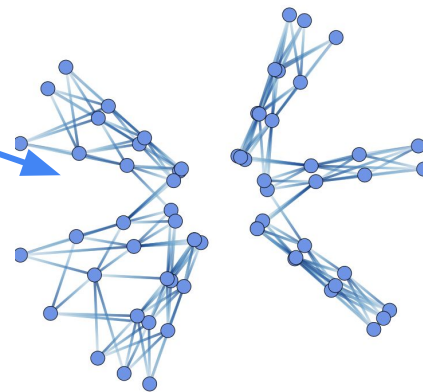
Ordered by p_T

Track 1 $p_T \quad \Delta\eta \quad \Delta\phi \quad d_0 \quad \dots$

Track 2 $p_T \quad \Delta\eta \quad \Delta\phi \quad d_0 \quad \dots$

$\vdots$

Track N $p_T \quad \Delta\eta \quad \Delta\phi \quad d_0 \quad \dots$



What's the best way to structure our data?

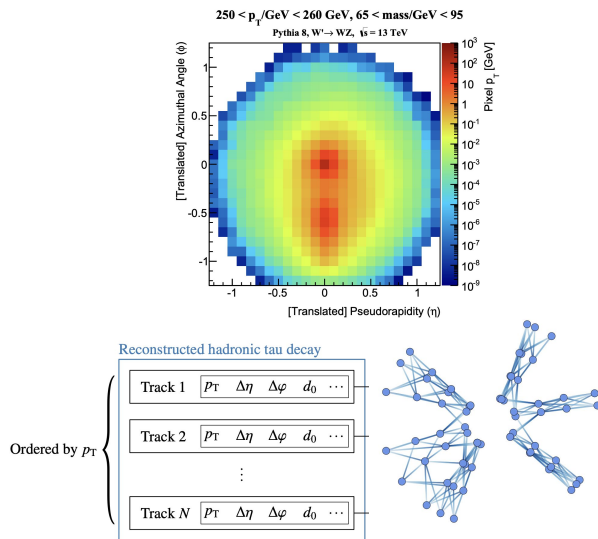
Prescribed structure

Learned structure

Industry-standard formats

Physics-informed

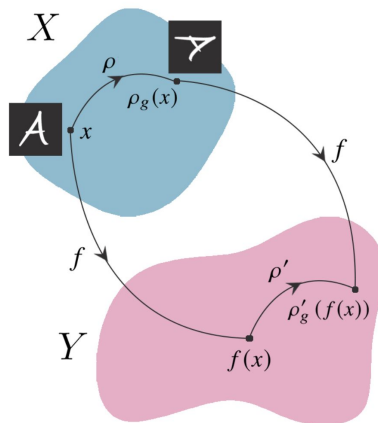
Self-supervised



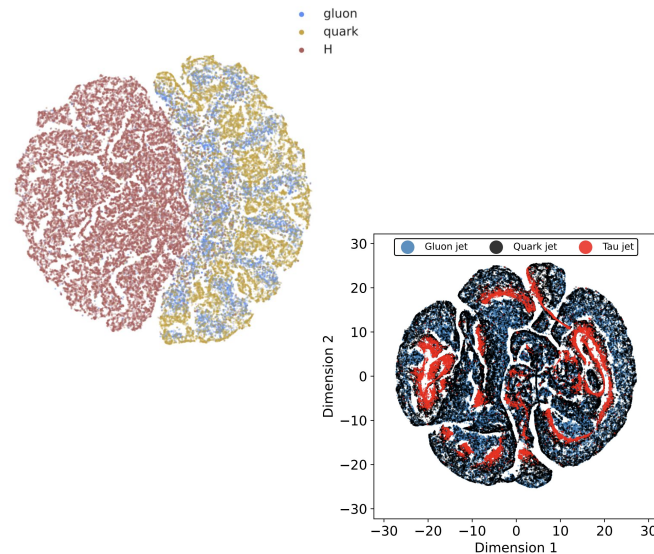
arXiv:1511.05190, arXiv:2007.13681

Equivariance

$$f(\rho_g(x)) = \rho'_g(f(x))$$



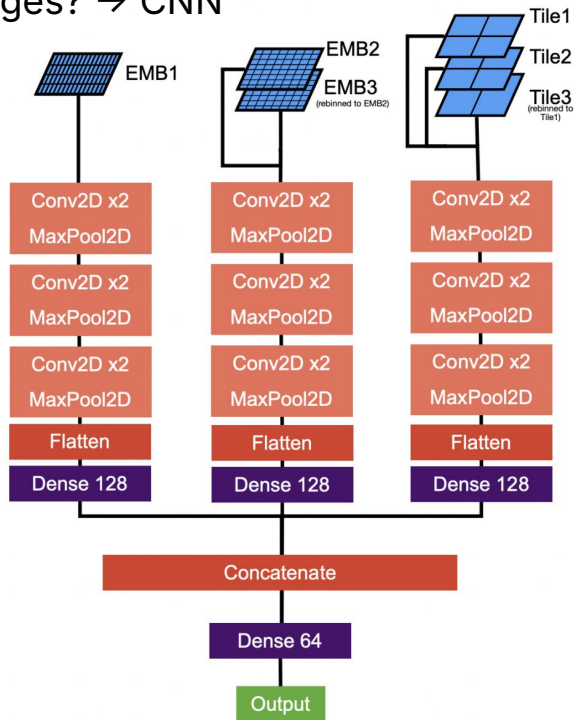
arXiv:2203.06153



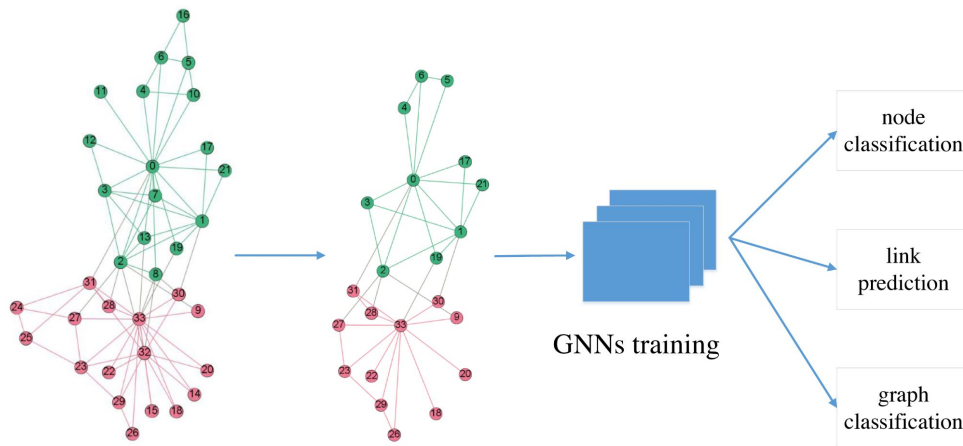
arXiv:2403.07066, arXiv:2503.11632

Geometric deep learning

Images? → CNN



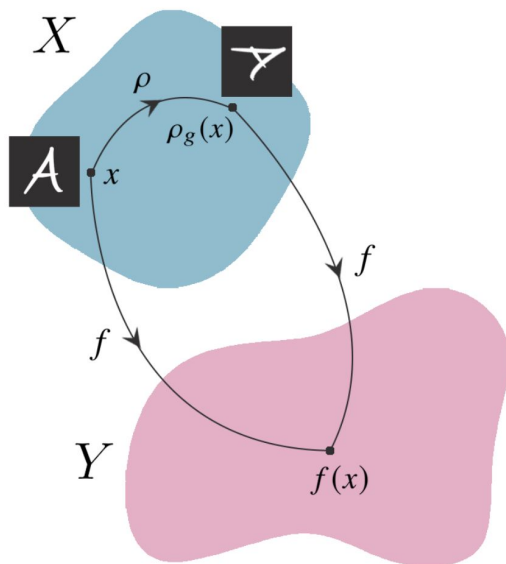
3D point cloud? → GNN or DeepSets



Geometric deep learning

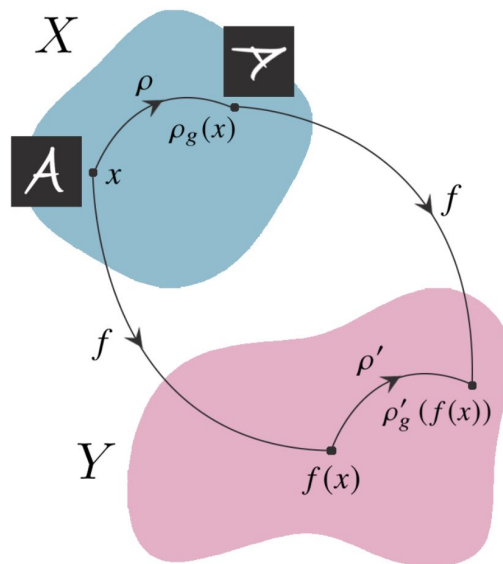
Invariance

$$f(\rho_g(x)) = f(x)$$



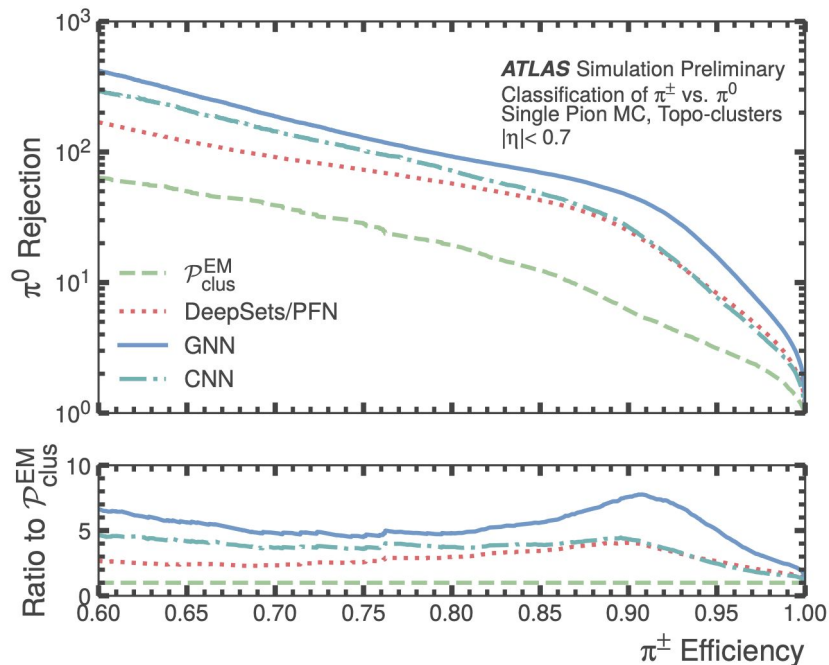
Equivariance

$$f(\rho_g(x)) = \rho'_g(f(x))$$



Geometric deep learning

Might see better performance / a wider range of input variables that can be included:



Can also be highly data efficient:

Training Fraction	Model	Accuracy	AUC	$1/\varepsilon_B$ ($\varepsilon_S = 0.5$)	$1/\varepsilon_B$ ($\varepsilon_S = 0.3$)
0.5%	ParticleNet	0.913	0.9687	77 ± 4	199 ± 14
	LorentzNet	0.929	0.9793	176 ± 14	562 ± 72
1%	ParticleNet	0.919	0.9734	103 ± 5	287 ± 19
	LorentzNet	0.932	0.9812	209 ± 5	697 ± 58
5%	ParticleNet	0.931	0.9807	195 ± 4	609 ± 35
	LorentzNet	0.937	0.9839	293 ± 12	1108 ± 84

Convolutional Neural Networks (CNNs)

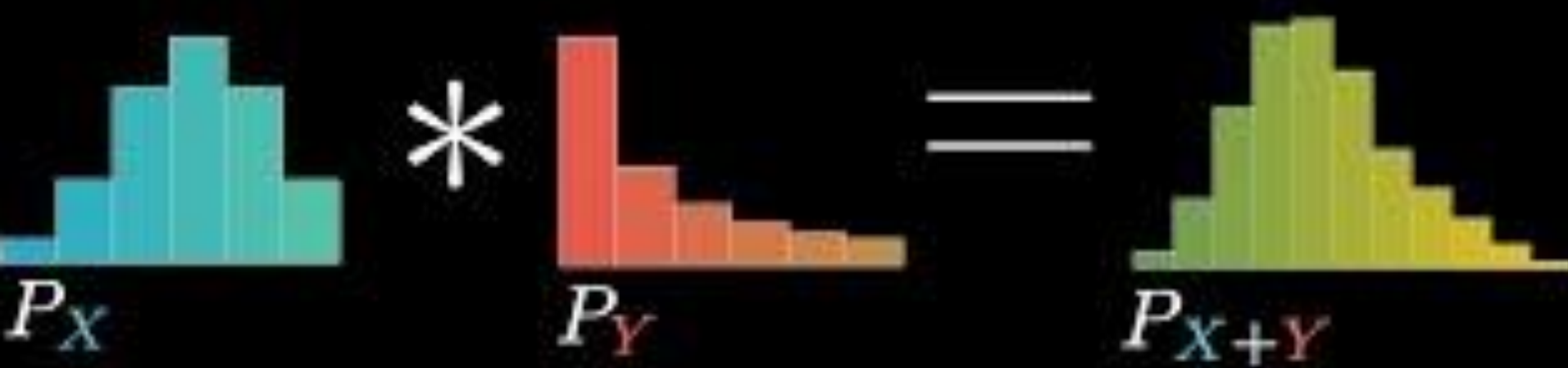
Convolutional Neural Networks

Convolution in 1D:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

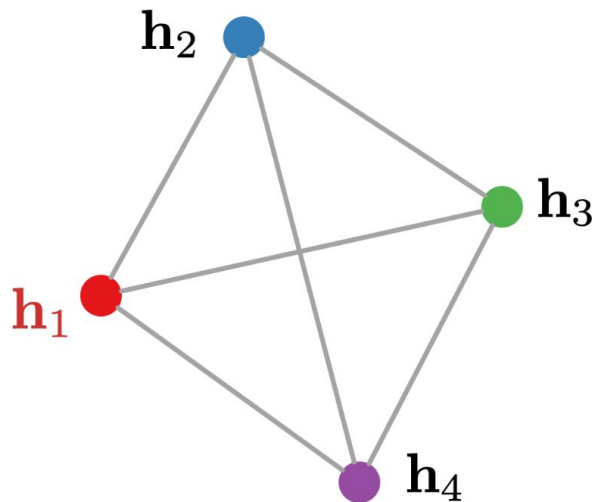
Convolution in 2D:

$$h_{i,j} = a \left[\beta + \sum_{m=1}^3 \sum_{n=1}^3 \omega_{m,n} x_{i+m-2,j+n-2} \right]$$

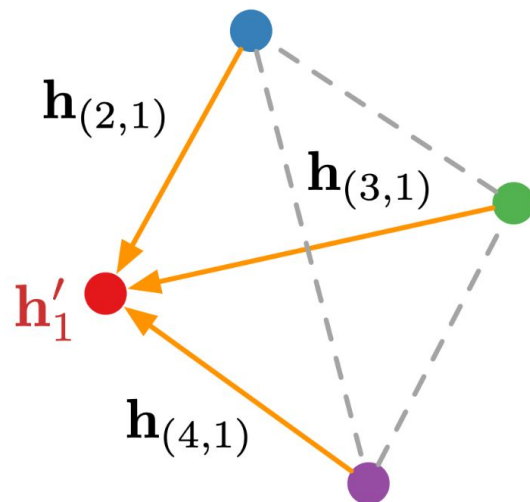


Graph Neural Networks (GNNs)

Graph Neural Networks



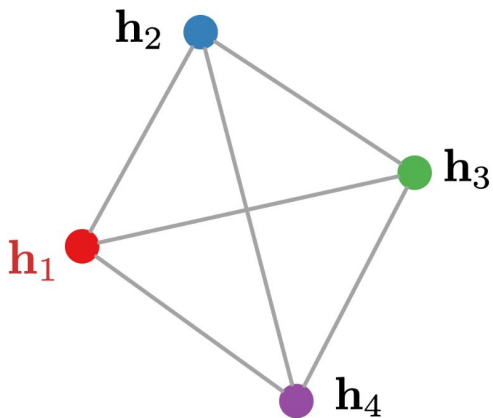
(a) Node representations.



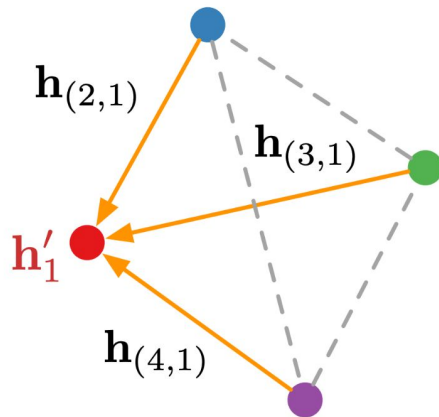
(b) Message passing step.

Graph Neural Networks

$$\mathbf{h}_{(i,j)} = f_{\text{edge}}(\mathbf{h}_i, \mathbf{h}_j, \mathbf{x}_{(i,j)}) ,$$
$$\mathbf{h}'_i = f_{\text{node}}(\mathbf{h}_i, \sum_{j \in \mathcal{N}_i} \mathbf{h}_{(j,i)}, \mathbf{x}_i)$$



(a) Node representations.



(b) Message passing step.

In-class work

Convolutions & CNNs:

<https://github.com/udlbook/udlbook/tree/main/Notebooks/Chap10>